

Cross-Domain Reasoning for Neuromorphic Model Design

Vikram Ramavarapu
vikramr2@illinois.edu
University of Illinois Urbana-Champaign
Urbana, Illinois, USA

Ashish Gautam
Oak Ridge National Laboratory
Oak Ridge, Tennessee, USA

Zachary Johnson-Scott
Oak Ridge National Laboratory
Oak Ridge, Tennessee, USA

Ramakrishnan Kannan
Oak Ridge National Laboratory
Oak Ridge, Tennessee, USA

Abstract

Designing performant neuromorphic models requires reasoning across neuroscience, neuromorphic computing, and machine learning, making it a natural target for cross-domain hypothesis generation. Our primary contribution is a multi-corpus knowledge graph spanning all three domains, which we show substantially increases cross-domain retrieval novelty over single-corpus baselines. We additionally introduce NeuKReAct, an agentic reasoning framework that iteratively retrieves from this graph and synthesizes design hypotheses via a step-by-step blackboard architecture, enabling structured compartmentalization of design decisions. Lastly, we introduce an execution head that translates hypotheses into structured design documents and runnable code. We evaluate novelty using a combinatorial creativity metric that measures cross-domain retrieval distance across the citation graph. Our results confirm that corpus breadth is the dominant driver of novelty. Moreover, we highlight a concrete instance of the novelty-utility tradeoff within NeuKReAct, underscoring a need for joint creativity evaluation, balancing both novelty and utility.

CCS Concepts

• **Computing methodologies** → **Semantic networks**; *Bio-inspired approaches*.

Keywords

Hypothesis generation, Knowledge graph reasoning, Neuromorphic computing, Biologically plausible learning, Automated scientific discovery

ACM Reference Format:

Vikram Ramavarapu, Zachary Johnson-Scott, Ashish Gautam, and Ramakrishnan Kannan. 2026. Cross-Domain Reasoning for Neuromorphic Model Design. In *International Green and Sustainable Computing Conference (IGSC 2026)*, June 22–24, 2026, Canandaigua, NY, USA. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3797248.3816054>

1 Acknowledgments

An award of computer time was provided by the ASCR Leadership Computing Challenge (ALCC) program. This research used

This paper is authored by an employee(s) of the United States Government and is in the public domain. Non-exclusive copying or redistribution is allowed, provided that the article citation is given and the authors and agency are clearly identified as its source. Request permissions from owner/author(s).

IGSC 2026, Canandaigua, NY, USA

2026. ACM ISBN 979-8-4007-2520-3/2026/06
<https://doi.org/10.1145/3797248.3816054>

resources of the Oak Ridge Leadership Computing Facility, a DOE Office of Science User Facility. This manuscript has been authored by UT-Battelle, LLC under Contract No. DE-AC05-00OR22725 with the US Department of Energy. The United States Government retains and the publisher, by accepting the article for publication, acknowledges that the United States Government retains a non-exclusive, paid-up, irrevocable, world-wide license to publish or reproduce the published form of this manuscript or allow others to do so, for United States Government purposes. The Department of Energy will provide public access to these results of federally sponsored research in accordance with the DOE Public Access Plan (<http://energy.gov/downloads/doe-public-access-plan>).

2 Introduction

Neuromorphic computing is an inherently interdisciplinary field, drawing from neuroscience, hardware engineering, and machine learning, seeking to build computational systems that operate according to brain-inspired principles [16, 18]. At its core are spiking neural networks (SNNs) [21], which can be deployed on energy-efficient neuromorphic hardware. Designing effective neuromorphic models is inherently interdisciplinary: it requires reasoning across neuroscience, traditional deep learning, and hardware constraints, and proceeds through iterative cycles of design, simulation, and refinement that differ substantially from conventional backpropagation-based workflows.

A growing concern is that contemporary neuromorphic algorithms have drifted toward machine learning conventions at the expense of neuroscientific grounding [14], underscoring the challenge of developing approaches that utilize a balance of insights across multiple domains.

The emergence of large language models (LLMs) and agentic reasoning frameworks opens a new avenue for addressing this complexity. LLMs can integrate knowledge spanning neuroscience, neuromorphic computing, and machine learning to reason about model design, while coding agent capabilities enable iterative implementation and testing in simulator environments. This suggests an LLM-driven design loop that could substantially reduce the expert effort required for neuromorphic model development. However, existing LLM and RAG approaches have not been widely applied to neuromorphic model design, and those targeting adjacent scientific domains retrieve within a single corpus rather than synthesizing across disciplinary boundaries [3, 25].

In this paper, we make three contributions:

- (1) **Multi-corpus knowledge graph** — an expansion of NeuKReAct’s original corpus to span neuroscience, traditional machine

learning, and neuromorphic computing, including code repositories, creating a unified graph across the domains relevant to neuromorphic model design. We show that this corpus breadth is the dominant driver of cross-domain retrieval novelty.

- (2) **NeuKReAct** — an agentic reasoning framework that iteratively retrieves from the multi-corpus graph and synthesizes neuromorphic model design hypotheses via a step-by-step blackboard architecture, enabling structured compartmentalization of design decisions that single-pass hypothesis generation cannot provide.
- (3) **Execution head** — both NeuKrag and NeuKReAct are extended with an execution head that produces a structured design document and starting codebase, enabling an off-the-shelf coding agent to implement each hypothesis end-to-end.

3 Background

3.1 Neuromorphic Computing and Spiking Neural Networks

Neuromorphic computing encompasses hardware architectures, simulation frameworks, and model design paradigms inspired by the structure and function of the brain. On the hardware side, neuromorphic chips such as Intel’s Loihi [8] and IBM’s TrueNorth [2] implement massively parallel, event-driven computation that achieves orders-of-magnitude improvements in energy efficiency over conventional processors for suitable workloads. Frameworks such as Fugu [1], Brian2 [30], and SuperNeuro [7] provide software environments for simulating and deploying neuromorphic models, while higher-level libraries like SNNtorch [10] integrate with modern deep learning pipelines to ease model development.

At the model level, spiking neural networks (SNNs) [21] are the central computational primitive of neuromorphic computing. Unlike traditional artificial neural networks that pass continuous activations, SNNs communicate through discrete spike events, enabling sparse, temporally-coded computation that maps naturally to neuromorphic hardware. Training SNNs is more challenging than conventional networks: the non-differentiable spike function requires approximate methods such as surrogate gradients [23, 29] for backpropagation-through-time, or local Hebbian rules [24], such as spike-timing-dependent plasticity (STDP) [28], wherein synaptic weights are adjusted based on the relative timing of pre- and post-synaptic spikes.

3.2 LLM Reasoning Paradigms

Large language models (LLMs) have demonstrated strong capabilities beyond language generation, including multi-step reasoning and decision-making in agentic settings [33]. Several reasoning paradigms, such as chain-of-thought prompting [35], have been developed to elicit these capabilities to encourage step-by-step reasoning. Tool calling further extends LLM agency by allowing the model to invoke external resources rather than relying solely on parametric knowledge [27]. Retrieval augmented generation (RAG) [19] is a specific form of tool use where the model retrieves relevant documents from an external corpus to inform its reasoning and generation.

ReAct [36] interleaves reasoning and acting steps in a single loop, making it particularly suited to tasks that require iterative information gathering and decision-making. The effectiveness of a ReAct agent depends heavily on the quality of its tool set and the relevance of the information it can retrieve. NeuKReAct exploits this by equipping the ReAct agent with graph-based retrieval over a multi-corpus knowledge base spanning neuromorphic computing, neuroscience, and traditional machine learning, enabling more grounded and cross-domain design reasoning.

A complementary idea from classical AI is the *blackboard architecture* [12], in which a shared working memory, the blackboard, holds the current state of a problem, and a set of independent knowledge sources read from and write to it as the solution evolves.

These agentic reasoning paradigms have been applied to scientific discovery [3, 25], but to the best of our knowledge, they have not been widely explored in the context of neuromorphic model design.

3.3 Combinatorial Creativity

Einstein famously remarked that “Combinatory play seems to be the essential feature in productive thought” [9]. This ‘combinatorial play’ is what Margaret Boden famously described as combinatorial creativity [6], where novel ideas emerge from the combination of existing concepts in new ways. More recently, combinatorial creativity scores have been proposed as a way to measure both creativity and generalization in LLMs [26].

Neuromorphic model design is a natural target for combinatorial creativity: producing good designs requires combining insights from neuroscience, hardware design, and machine learning in non-obvious ways. The challenge is structural, as we show in Section 4.1, these three domains infrequently cite one another in practice, meaning cross-domain synthesis is both rare and genuinely creative when it occurs. We adopt combinatorial creativity as our primary evaluation lens, measuring whether our frameworks retrieve papers from domains that are distant in the citation graph.

3.4 NeuKrag

NeuKrag [11] is a non-agentic framework for neuromorphic model design that uses a single retrieval step over a knowledge graph, inferred from a corpus of neuromorphic computing literature. Its hypothesis generation is a single-pass response: all design decisions are produced at once, without iterative refinement or compartmentalization across design stages. While NeuKrag demonstrated the potential of LLM-based retrieval for neuromorphic model design, it was limited to a single domain and lacked planning or implementation capability. In this work, we address these two limitations independently: the multi-corpus expansion broadens the retrieval substrate to span neuroscience and machine learning alongside neuromorphic computing, while NeuKReAct adds structured planning through a ReAct loop and step-by-step blackboard architecture.

4 Data

4.1 Publication Corpus

Our new publication corpus consists of 3,658 papers drawn from three domains: 750 neuroscience, 1,163 traditional machine learning, and 1,745 neuromorphic computing, with 8,771 citation events.

Corpus sizes reflect the outcome of construction rather than pre-determined targets; our goal was balanced representation across domains. The corpus exhibits striking domain isolation: only 5 citation events bridge neuroscience and ML, and just 1 bridges ML and neuromorphic computing, an empirical property of the literature, not a design choice, and precisely the gap our multi-corpus expansion addresses. Each domain corpus was assembled independently using the CrossRef API, then merged; the neuroscience and AI/ML corpora were validated against 10 marker papers spanning seminal works (Hodgkin & Huxley [13], Izhikevich [15], Bi & Poo [5], Vaswani et al. [32], Mirza & Osindero [22]).

Neuromorphic computing. Reused directly from NeuKrag. The citation network was clustered with the Leiden algorithm [31] (resolution 0.0001) to retain one large core cluster and discard peripheral noise, yielding 1,745 papers.

Neuroscience. Seeded from 86,958 BioRxiv papers tagged “neuroscience” (zero initial citation overlap with neuromorphic). Papers citing both the seed corpus and the neuromorphic corpus were identified via CrossRef, then expanded two hops out in the neuroscience citation network. The resulting 750 papers carry 202 citation events to the neuromorphic corpus.

Traditional machine learning. Seeded from 124,446 papers across two HuggingFace datasets (“ML-ArXiv-Papers” and “ICML-Accepted-Papers”). CrossRef and SemanticScholar APIs were used jointly to handle the high prevalence of preprints. A Steiner tree was computed over the citation network connecting two terminal sets: the AI/ML marker papers and all papers with at least one citation event to the other two corpora. The tree was then expanded by including all Leiden-cluster neighbors of each Steiner node (resolution 0.01, chosen empirically from {0.1, 0.01, 0.001, 0.0001} to match the scale of the other corpora), yielding 1,163 papers with 5 citation events to neuroscience and 1 to neuromorphic computing.

4.2 Code Corpus

The code corpus consists of GitHub repositories covering `snnTorch`, `Brian2`, `Fugu`, and `SuperNeuro`, collected via the GitHub search API. Each repository is indexed at the chunk level (split at class and function boundaries) into a flat JSONL file, with official web documentation or AST-extracted API summaries appended per library.

5 Model Architecture

NeuKReAct is implemented in DSPy [17] and is organized around three modular components: a *graph-based corpus*, a *hypothesis agent* that performs combinatorial hypothesis generation over the multi-corpus knowledge graph, and an *execution head* that first generates a starting repository with a structured design document and codebase, then deploys an off-the-shelf coding agent for iterative implementation and testing.

5.1 Graph-based Corpus

NeuKReAct fetches from a single in-memory store, `NeuroGraphStore`. All records are keyed on a `domain/paper_id` composite identifier, where `domain` is one of `aiml`, `neuroscience`, or `neuromorphic`, and `paper_id` is the numeric index assigned to each paper during corpus construction. Loading proceeds in two passes per domain:

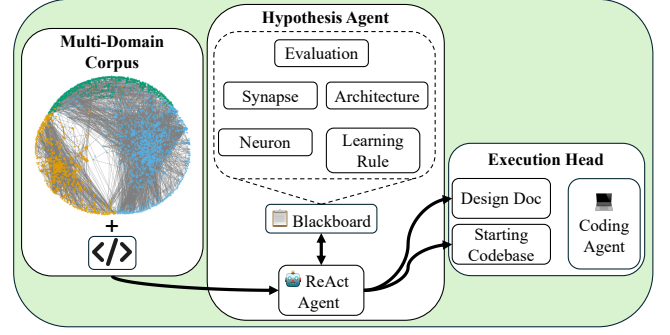


Figure 1: NeuKReAct architecture. The agent consists of three components: a corpus, a hypothesis agent, and an execution head. Each of these components are modular and swappable.

first, the citation graph is reconstructed from node and edge CSV files as a NetworkX directed graph, with author and year meta-data extracted from the node table; second, a per-domain JSON file is read to populate paper nodes and section chunks, with the author and year fields joined in from the citation graph metadata. Equations are extracted inline during this second pass by matching display-math LaTeX patterns in section bodies. The code index is loaded independently from the pre-built JSONL file.

For semantic retrieval, all three content types are embedded offline using `all-MiniLM-L6-v2` [34] and stored as FAISS IndexFlatIP indices. Separate indices are maintained per domain for sections, and globally for equations and code. At query time the store dispatches to dense vector search when indices are loaded, falling back to weighted keyword frequency scoring otherwise.

We also used NeuKrag’s KG assembly pipeline on our expanded multi-corpus dataset to produce a larger knowledge graph to enable multi-corpus KG-RAG-based hypothesis generation. As such, this corpus can be swapped out for the original NeuKrag KG, both across its original single-domain and a new multi-domain version that we constructed.

5.2 Hypothesis Agent

In NeuKReAct, the hypothesis agent is a `dspy.ReAct` module equipped with eleven retrieval tools: full-text and vector section search, equation retrieval, cross-domain link discovery, citation neighborhood traversal, code search and file reading, and pre-built library API references. The agent executes a locked seven-stage pipeline — *lit_search*, *neuron_choice*, *synapse_choice*, *learning_rule*, *architecture*, *implementation*, *evaluation*, committing each decision to a persistent markdown *blackboard* before advancing.

This hypothesis agent is designed to be swappable and modular, allowing us to swap between NeuKReAct and NeuKrag’s original KG-RAG-based agent to evaluate the impact of iterative reasoning on creativity and utility.

In *QnA mode*, the pipeline stops here, as the agent answers free-form research questions using the same retrieval tools without needing to proceed to the execution head.

5.3 Execution Head

Once the ReAct agent completes all stages, the agent produces a validated workspace: five files (`network.py`, `train.py`, `evaluate.py`, `requirements.txt`, `check.py`) are generated and verified by running `check.py` until it passes.

In our work, the execution head first applies an in-house translation step that uses the hypothesis agent’s already-retrieved papers and design decisions to generate an initial codebase grounded in the relevant literature, ensuring the starting repository reflects the cross-domain insights surfaced during hypothesis generation. A coding agent subprocess then receives this grounded workspace and runs `train.py` and `evaluate.py` end-to-end, fixing errors and tuning for accuracy autonomously. The ReAct agent then reads a `changes_summary.md` and all modified files to reconcile the blackboard with the real implementation before producing a Neuro-morphic Design Report saved as `design_spec.md`. Because the execution head is a separate, swappable component, the hypothesis generation pipeline can be evaluated independently of any particular coding agent; it is only by attaching an execution head that the utility checks described in Section 6.1 become observable, which in turn makes the full creativity score computable.

In our work, the hypothesis agent, design doc and initial codebase generation is performed by OpenAI gpt-oss-120B, while the coding agent is Claude Code running Sonnet 4.6.

6 Evaluation

6.1 Creativity

We evaluate the cross-domain creativity of our systems by adapting the combinatorial creativity score of Schapiro et al. [26], which defines creativity as the product of novelty and utility. In their formulation, novelty is measured as path length and label surprise on a synthetic conceptual graph, and utility is measured as constraint satisfaction. We adapt this framework to our setting, where the conceptual space is the citation graph described in Section 4.1, and the creative artifacts are the sets of papers the hypothesis agent retrieves and cites when producing a design.

Novelty. A design output is novel to the extent that it draws on papers from domains that are distant in the citation graph. Concretely, given the set of papers $R = \{p_1, \dots, p_k\}$ retrieved by the hypothesis agent during a design session, we define novelty as the mean shortest-path distance between all pairs of retrieved papers in the citation graph:

$$N(R) = \frac{1}{\binom{k}{2}} \sum_{i < j} d(p_i, p_j) \quad (1)$$

where $d(p_i, p_j)$ is the shortest-path distance between papers p_i and p_j in the citation graph. A high novelty score indicates that the hypothesis agent is combining insights from papers that are not typically cited together, reflecting genuine cross-domain reasoning. Since NeuKrag uses a knowledge graph instead of a citation graph, we use KG triple provenance to map retrieved facts back to their source papers and compute $N(R)$ in the same way.

Utility. Downstream utility can be represented as the product of the faithfulness of an implementation to its hypothesis and the functionality of that implementation. Utility is what grounds

Table 1: Biological inspirations provided to each system per task. Each system must use at least two in its design.

#	N-MNIST Classification	Inverted Pendulum RL
1	Visual cortex hierarchy (V1/V2)	Hippocampal sequence memory
2	Lateral inhibition	Working memory / pre-frontal cortex
3	Temporal coding / spike timing	Dopaminergic reward signals (basal ganglia)
4	Hebbian learning	Cerebellar error correction

novelty: without it, the novelty metric risks rewarding mere retrieval diversity rather than scientifically meaningful cross-domain synthesis. In this work, we report functionality via two binary necessary-condition checks: whether the generated code runs without error (*Runs*), and whether training loss decreases over at least one epoch (*Learns*). These gates establish whether a hypothesis is implementable in principle, i.e. a design that fails either cannot be meaningfully evaluated on downstream performance metrics. Full utility scoring, including accuracy, spike sparsity, and biological plausibility, is reserved for future work.

Creativity score. Following Schapiro et al. [26], creativity is the product of novelty and utility:

$$C(R) = N(R) \cdot U \quad (2)$$

In this paper, we report $N(R)$ as the primary evaluation signal. The full creativity score C requires systematic utility evaluation and is deferred to future work.

6.2 Experimental Setup

In order to create a controlled environment in which agents are primed to search across domains, each system is prompted with a brief task description and a set of four biological inspirations drawn from neuroscience and neuromorphic computing literature, of which it must use at least two in its design. The inspirations used are listed in Table 1. To evaluate implementability, each system that produces a design document is passed to Claude Code to implement in SNN-Torch. We record whether the resulting code runs without error and whether training loss decreases over at least one epoch. Novelty $N(R)$ is computed directly from the set of papers retrieved during the design session, independent of implementation.

We evaluate on two tasks: **N-MNIST classification** and **MuJoCo inverted pendulum control**. The six systems compared are summarized in Table 2.

Novelty scores and implementability results are reported in Table 3. NeuKrag and NeuKrag-inter produce text responses only and are not passed to an execution head, so *Runs* and *Learns* are not applicable for these systems. NeuKrag-code and NeuKrag-inter-code attach the execution head, producing refined, runnable implementations. Systematic performance benchmarking is reserved for future work.

Table 2: Systems compared in evaluation. “Exec. head” indicates whether a coding agent is attached.

System	Corpus	Agent	Exec. head
NeuKReAct	Multi	ReAct	✓
NeuKrag-code	Single	KG-RAG	✓
NeuKrag-inter-code	Multi	KG-RAG	✓
NeuKrag	Single	KG-RAG	×
NeuKrag-inter	Multi	KG-RAG	×
Claude Code	—	—	✓

Table 3: Novelty scores and implementability results across systems and tasks. $N(R)$ is the mean shortest-path distance between retrieved papers in the citation graph; higher values indicate more cross-domain retrieval. Runs and Learns are binary implementability checks; N/A indicates systems without an execution head for which these checks are not applicable. [‡]Claude Code performs no retrieval from the citation network, so $N(R) = 0$ by definition.

Model	Task	$N(R)$	Runs	Learns
NeuKReAct	N-MNIST	5.39	✓	✓
NeuKrag-code		4.00	✓	✓
NeuKrag-inter-code		7.13	✓	✓
NeuKrag		4.00	N/A	N/A
NeuKrag-inter		7.13	N/A	N/A
Claude Code [‡]		0	✓	✓
NeuKReAct	Pendulum	1	✓	✓
NeuKrag-code		3.76	✓	✓
NeuKrag-inter-code		4.80	✓	✓
NeuKrag		3.76	N/A	N/A
NeuKrag-inter		4.80	N/A	N/A
Claude Code [‡]		0	✓	✓

7 Results

Novelty. Table 3 shows that multi-corpus systems consistently achieve higher $N(R)$ values, confirming that corpus breadth is a primary driver of cross-domain retrieval novelty. NeuKrag-inter and NeuKrag-inter-code achieve the highest $N(R)$ on both tasks, likely because the knowledge graph provides a more general abstraction that maps to a wider range of papers. NeuKReAct scores $N(R) = 1$ on the pendulum task, as the agent retrieved a single neuroscience paper on eligibility propagation [4] and snnTorch reference documentation and found those sufficient to proceed directly to implementation. Notably, a separate rerun of NeuKReAct on the pendulum task achieved $N(R) = 8$ by retrieving a substantially more diverse set of cross-domain papers. However, the execution head for that run produced a model that could only balance the pendulum for 20 frames, compared to 500 frames for the $N(R) = 1$ run. Table 4 compares the two runs; the higher-novelty design synthesized a more architecturally complex hypothesis drawing from multiple domains, while the lower-novelty run converged on a targeted, principled learning rule that proved more tractable for the execution head. This directly illustrates the novelty-utility tradeoff identified by Schapiro et al. [26]: a higher $N(R)$ does not

guarantee a more useful design, and underscores the run-instance sensitivity of the current system.

Table 4: Comparison of two NeuKReAct runs on the Pendulum task, illustrating the novelty-utility tradeoff.

	$N(R) = 8$ run	$N(R) = 1$ run
Papers retrieved	2 cross-domain	1 + library ref
Code fetched	snnTorch API (RLeaky)	Spiking cerebellum impl. + snnTorch API
Learning rule	R-Hebb + elig. traces	e-prop [4]
Architecture	Hippocampal buf. + cerebellar fwd. model	Actor-critic + cerebellum
Frames balanced	20	500

Implementability. All systems with an execution head produced runnable code that was capable of learning. Moreover, Claude Code alone was capable of producing runnable, learnable code without any retrieval from the citation graph. This suggests that while it may be true that the design of the design document and code-base generation module could be effective at gluing the hypothesis agents to the coding agent, the coding agent alone is enough to bring any of our generated hypotheses to base-level working implementations. Alongside the various degrees of performance achieved by the execution head as shown in Table 4, this suggests that baseline implementability is not a major bottleneck for creativity in our current system, and that the primary challenge is generating hypotheses that are both novel and useful enough to produce high-performing models.

Hypothesis structure. Beyond the novelty score, NeuKReAct’s blackboard architecture produces more structured outputs than NeuKrag-inter’s single-pass response. For the Pendulum task, NeuKReAct committed each design decision to a dedicated stage: a per-paper literature table with explicit domain labels and contributions, a formal learning-rule specification (e.g., the eligibility-trace update), and an architecture diagram, all as separate, inspectable entries. NeuKrag collapsed the same design space into a single prose block where neuron model, learning rule, and architecture were described together with no stage boundaries, making it difficult to trace which retrieved paper motivated which design choice. This compartmentalization is NeuKReAct’s primary advantage.

Overall, corpus breadth is the most consistent driver of retrieval novelty. Our rerun finding — where a higher-novelty NeuKReAct design produced a worse-performing model — concretely motivates the full creativity score $C = N(R) \cdot U$ as the necessary evaluation target, rather than novelty alone.

8 Limitations and Future Work

As this work focuses on software-level model design, extending NeuKReAct to hardware architecture and fabrication is an important open direction. Our results establish corpus breadth as the primary novelty driver, but the planning contribution of NeuKReAct’s ReAct loop and blackboard architecture has not yet been evaluated against a utility metric that captures downstream task performance; developing stronger utility metrics that link novelty to task outcomes is therefore a key open direction. Moreover, while

the corpus significantly improves agentic creativity, balancing the corpus to hold even representation across domains is an open challenge. Broader library support beyond `snnTorch`, and evaluation under constrained local learning rules such as STDP and Hebbian learning, remain open. Finally, multi-agent architectures [20] that distribute reasoning across domain specialists may better capture the diversity of expertise that neuromorphic model design demands. Lastly, future work entailing more rigorous creativity evaluation calls for tasks complex enough to require multi-component designs and open-ended enough that the standard ML framework of architecture search and accuracy optimization is insufficient. A key metric limitation is that code retrieval is currently invisible to the citation-graph novelty score; extending the unified knowledge graph to include code repository nodes would allow code fetches to contribute to $N(R)$ and give a more complete picture of cross-domain grounding.

References

- [1] James B Aimone, William Severa, and Craig M Vineyard. 2019. Composing Neural Algorithms with Fugu. *arXiv:1905.12130* [cs.NE] <https://arxiv.org/abs/1905.12130>
- [2] Filipp Akopyan, Jun Sawada, Andrew Cassidy, Rodrigo Alvarez-Icaza, John Arthur, Paul Merolla, Nabil Imam, Yutaka Nakamura, Pallab Datta, Gi-Joon Nam, Brian Taba, Michael Beakes, Bernard Brezzo, Jente B Kuang, Rajit Manohar, William P Risk, Bryan Jackson, and Dharmendra S Modha. 2015. TrueNorth: Design and tool flow of a 65 mW 1 million neuron programmable neurosynaptic chip. *IEEE Trans. Comput.-aided Des. Integr. Circuits Syst.* 34, 10 (Oct. 2015), 1537–1557.
- [3] Akari Asai, Jacqueline He, Rulin Shao, Weijia Shi, Amanpreet Singh, Joseph Chee Chang, Kyle Lo, Luca Soldaini, Sergey Feldman, Mike D’Arcy, David Wadden, Matt Latzke, Jenna Sparks, Jena D Hwang, Varsha Kishore, Minyang Tian, Pan Ji, Shengyan Liu, Hao Tong, Bohao Wu, Yanyu Xiong, Luke Zettlemoyer, Graham Neubig, Daniel S Weld, Doug Downey, Wen-Tau Yih, Pang Wei Koh, and Hannaneh Hajishirzi. 2026. Synthesizing scientific literature with retrieval-augmented language models. *Nature* 650, 8103 (Feb. 2026), 857–863.
- [4] Guillaume Bellec, Franz Scherr, Anand Subramoney, Elias Hajek, Darjan Salaj, Robert Legenstein, and Wolfgang Maass. 2020. A solution to the learning dilemma for recurrent networks of spiking neurons. *Nat. Commun.* 11, 1 (July 2020), 3625.
- [5] G Q Bi and M M Poo. 1998. Synaptic modifications in cultured hippocampal neurons: dependence on spike timing, synaptic strength, and postsynaptic cell type. *J. Neurosci.* 18, 24 (Dec. 1998), 10464–10472.
- [6] Margaret A. Boden. 2004. *The Creative Mind: Myths and Mechanisms*. Routledge.
- [7] Prasanna Date, Chathika Gunaratne, Shruti Kulkarni, Robert Patton, Mark Coletti, and Thomas Potok. 2023. SuperNeuro: A Fast and Scalable Simulator for Neuromorphic Computing. *arXiv:2305.02510* [cs.NE] <https://arxiv.org/abs/2305.02510>
- [8] Mike Davies, Andreas Wild, Garrick Orchard, Yulia Sandamirskaya, Gabriel A Fonseca Guerra, Prasad Joshi, Philipp Plank, and Suredh R Risbud. 2021. Advancing neuromorphic computing with loihi: A survey of results and outlook. *Proc. IEEE Inst. Electr. Electron. Eng.* 109, 5 (May 2021), 911–934.
- [9] Albert Einstein, Carl Seelig, Sonja Bargmann, Issachar Unna, and Barbara Wolff. 1954. Ideas and opinions. (1954).
- [10] Jason K. Eshraghian, Max Ward, Emre Neftci, Xinxin Wang, Gregor Lenz, Girish Dwivedi, Mohammed Bannamoun, Doo Seok Jeong, and Wei D. Lu. 2023. Training Spiking Neural Networks Using Lessons From Deep Learning. *arXiv:2109.12894* [cs.NE] <https://arxiv.org/abs/2109.12894>
- [11] Ashish Gautam, Robert Patton, Thomas Potok, Ramakrishnan Kannan, James Aimone, and William Severa. 2025. AI-Powered Knowledge Graphs for Neuromorphic and Energy-Efficient Computing. In *Proceedings of the Great Lakes Symposium on VLSI 2025 (GLSVLSI ’25)*. Association for Computing Machinery, New York, NY, USA, 996–1001. doi:10.1145/3716368.3735295
- [12] Bochen Han and Songmao Zhang. 2025. Exploring Advanced LLM Multi-Agent Systems Based on Blackboard Architecture. *arXiv:2507.01701* [cs.MA] <https://arxiv.org/abs/2507.01701>
- [13] A L Hodgkin and A F Huxley. 1952. A quantitative description of membrane current and its application to conduction and excitation in nerve. *J. Physiol.* 117, 4 (Aug. 1952), 500–544.
- [14] Giacomo Indiveri. 2025. Neuromorphic is dead. Long live neuromorphic. *Neuron* 113 (2025), 3311–3314. doi:10.1016/j.neuron.2025.09.020
- [15] E M Izhikevich. 2003. Simple model of spiking neurons. *IEEE Trans. Neural Netw.* 14, 6 (2003), 1569–1572.
- [16] Benedikt Jung, Maximilian Kalcher, Merlin Marinova, Piper Powell, and Esma Sakalli. 2025. Neuromorphic Computing - An Overview. *arXiv:2510.06721* [cs.NE] <https://arxiv.org/abs/2510.06721>
- [17] Omar Khattab, Arnab Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhaman, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2023. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. *arXiv:2310.03714* [cs.CL] <https://arxiv.org/abs/2310.03714>
- [18] Dhireesha Kudithipudi, Catherine Schuman, Craig M Vineyard, Tej Pandit, Cory Merkel, Rajkumar Kubendran, James B Aimone, Garrick Orchard, Christian Mayr, Ryad Benosman, Joe Hays, Cliff Young, Chiara Bartolozzi, Amitava Majumdar, Suma George Cardwell, Melika Payvand, Sonia Buckley, Shruti Kulkarni, Hector A Gonzalez, Gert Cauwenberghs, Chetan Singh Thakur, Anand Subramoney, and Steve Furber. 2025. Neuromorphic computing at scale. *Nature* 637, 8047 (Jan. 2025), 801–812.
- [19] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2021. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *arXiv:2005.11401* [cs.CL] <https://arxiv.org/abs/2005.11401>
- [20] Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujia Yang, Shuming Shi, and Zhaopeng Tu. 2024. Encouraging Divergent Thinking in Large Language Models through Multi-Agent Debate. *arXiv:2305.19118* [cs.CL] <https://arxiv.org/abs/2305.19118>
- [21] Kai Malcolm and Josue Casco-Rodriguez. 2023. A Comprehensive Review of Spiking Neural Networks: Interpretation, Optimization, Efficiency, and Best Practices. *arXiv:2303.10780* [cs.NE] <https://arxiv.org/abs/2303.10780>
- [22] Mehdi Mirza and Simon Osindero. 2014. Conditional Generative Adversarial Nets. *arXiv:1411.1784* [cs.LG] <https://arxiv.org/abs/1411.1784>
- [23] Emre O. Neftci, Hesham Mostafa, and Friedemann Zenke. 2019. Surrogate Gradient Learning in Spiking Neural Networks. *arXiv:1901.09948* [cs.NE] <https://arxiv.org/abs/1901.09948>
- [24] Alexander Ororbia, Ankur Mali, Adam Kohan, Beren Millidge, and Tommaso Salvatori. 2024. A Review of Neuroscience-Inspired Machine Learning. *arXiv:2403.18929* [cs.NE] <https://arxiv.org/abs/2403.18929>
- [25] Yuanhao Qu, Kaixuan Huang, Ming Yin, Kanghong Zhan, Dyllan Liu, Di Yin, Henry C Cousins, William A Johnson, Xiaotong Wang, Mihir Shah, Russ B Altman, Denny Zhou, Mengdi Wang, and Le Cong. 2026. CRISPR-GPT for agentic automation of gene-editing experiments. *Nat. Biomed. Eng.* 10, 2 (Feb. 2026), 245–258.
- [26] Samuel Schapiro, Sumuk Shashidhar, Alexi Gladstone, Jonah Black, Royce Moon, Dilek Hakkani-Tur, and Lav R. Varshney. 2026. Combinatorial Creativity: A New Frontier in Generalization Abilities. *arXiv:2509.21043* [cs.AI] <https://arxiv.org/abs/2509.21043>
- [27] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. *arXiv:2302.04761* [cs.CL] <https://arxiv.org/abs/2302.04761>
- [28] Harel Z Shouval, Samuel S-H Wang, and Gayle M Wittenberg. 2010. Spike timing dependent plasticity: a consequence of more fundamental learning rules. *Front. Comput. Neurosci.* 4 (July 2010).
- [29] Sumit Bam Shrestha and Garrick Orchard. 2018. SLAYER: Spike Layer Error Reassignment in Time. *arXiv:1810.08646* [cs.NE] <https://arxiv.org/abs/1810.08646>
- [30] Marcel Stimberg, Romain Brette, and Dan Fm Goodman. 2019. Brian 2, an intuitive and efficient neural simulator. *Elife* 8, e47314 (Aug. 2019).
- [31] V A Traag, L Waltman, and N J van Eck. 2019. From Louvain to Leiden: guaranteeing well-connected communities. *Sci. Rep.* 9, 1 (March 2019), 5233.
- [32] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2023. Attention Is All You Need. *arXiv:1706.03762* [cs.CL] <https://arxiv.org/abs/1706.03762>
- [33] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science* 18, 6 (March 2024). doi:10.1007/s11704-024-40231-1
- [34] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. *arXiv:2002.10957* [cs.CL] <https://arxiv.org/abs/2002.10957>
- [35] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2023. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *arXiv:2201.11903* [cs.CL] <https://arxiv.org/abs/2201.11903>
- [36] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv:2210.03629* [cs.CL] <https://arxiv.org/abs/2210.03629>